

COMPUTER VISION · YOLOv11 · STREAMLIT

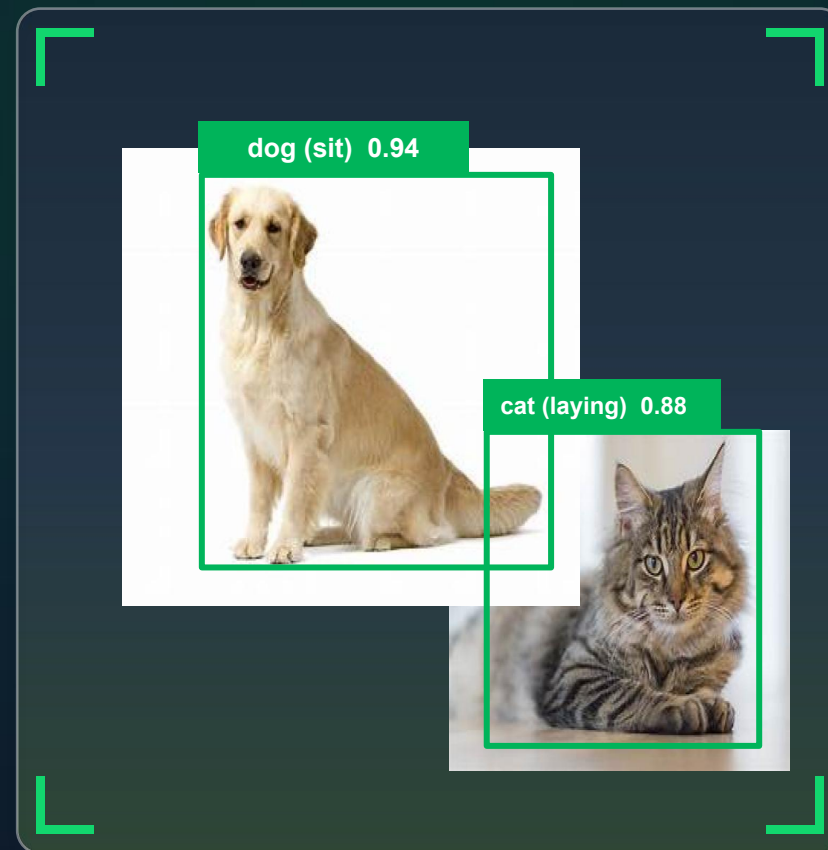
VisionBox

AI-Powered **Animal & Pose** Detection

Detect what an animal is — and what it's doing — from a single image, using a fine-tuned YOLOv11 model wrapped in a simple web app.

A beginner-friendly Computer Vision project

github.com/devMonkRahul/VisionBox



What is VisionBox?

VisionBox is a hands-on Computer Vision app that lets you upload a photo and instantly see which animals are in it, the pose each one is in, and how confident the model is — all through a clean Streamlit web interface, with no training required from the user.



Made for beginners

Designed for engineering students learning OpenCV for the first time.



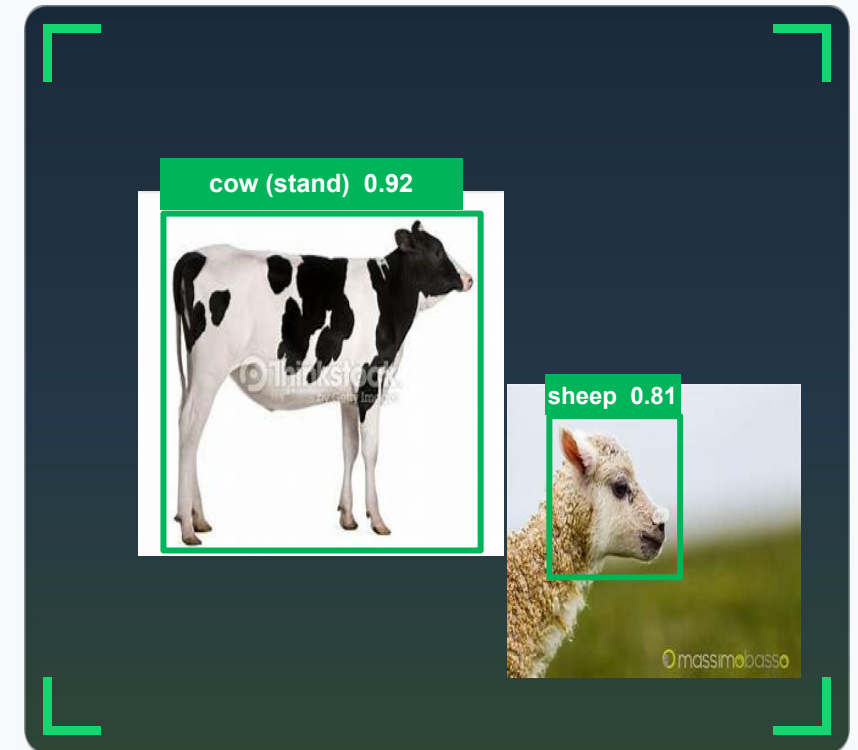
Workshop

Run the app, upload an image, and understand detection end-to-end in one session.



Fine-tuned YOLOv11

Uses a YOLOv11 nano model adapted to a custom animals dataset — ready to run.



One photo in → labelled animals + poses out

Computer Vision & Object Detection

Object detection answers two questions at once: what is in the image, and where is it. That's the core idea VisionBox demonstrates.

Image Classification

One label for the whole image



"cat"

- ✓ Answers "what is this?"
- ✓ No location information
- ✓ One prediction per image



Object Detection

Boxes + labels for every object

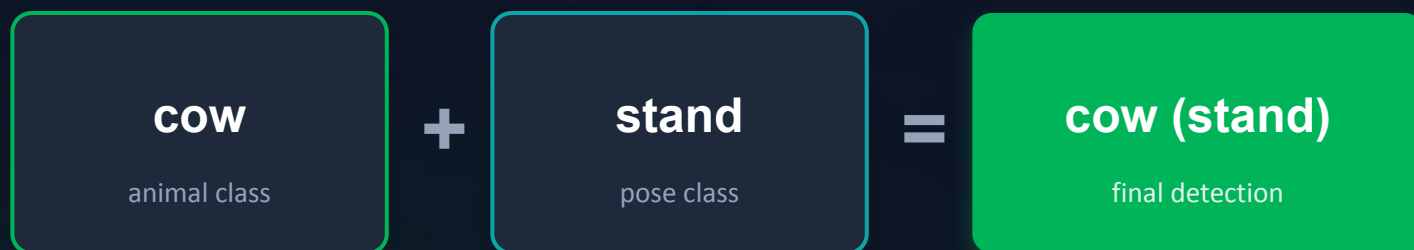


- ✓ Answers "what AND where?"
- ✓ Draws a bounding box per object
- ✓ Many objects, each with a score

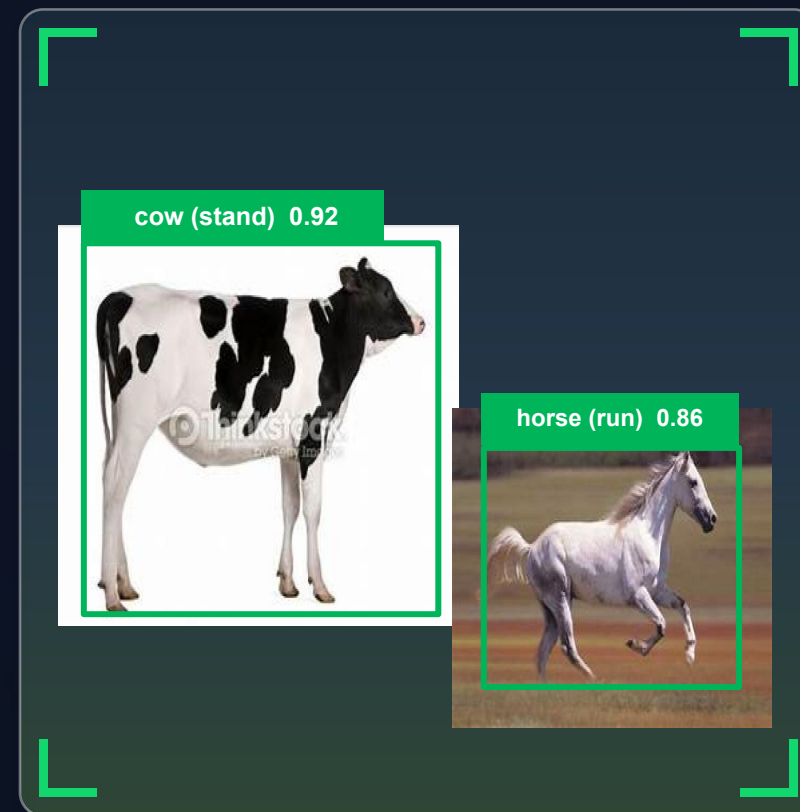
WHAT MAKES IT DIFFERENT

The Big Idea: Animal + Pose

Most detectors stop at “there is a cow.” VisionBox goes one step further: it detects the animal and what it is doing, then merges them into a single, human-readable label.



Each pose is matched to the animal whose box it overlaps the most, so the description stays accurate even with several animals in frame.



Animal + pose, paired automatically

The Technology Stack

VisionBox stays lightweight — a small, well-known Python stack that students can install and read in a single sitting.



Ultralytics YOLOv11

The detection engine — loads animals.pt and runs inference.



Streamlit

Builds the entire web UI: upload, sliders, tables, downloads.



OpenCV

Draws bounding boxes, labels and confidence onto the image.



pandas

Turns raw detections into clean tables and statistics.



NumPy

Converts images to arrays and handles box math (IoU).



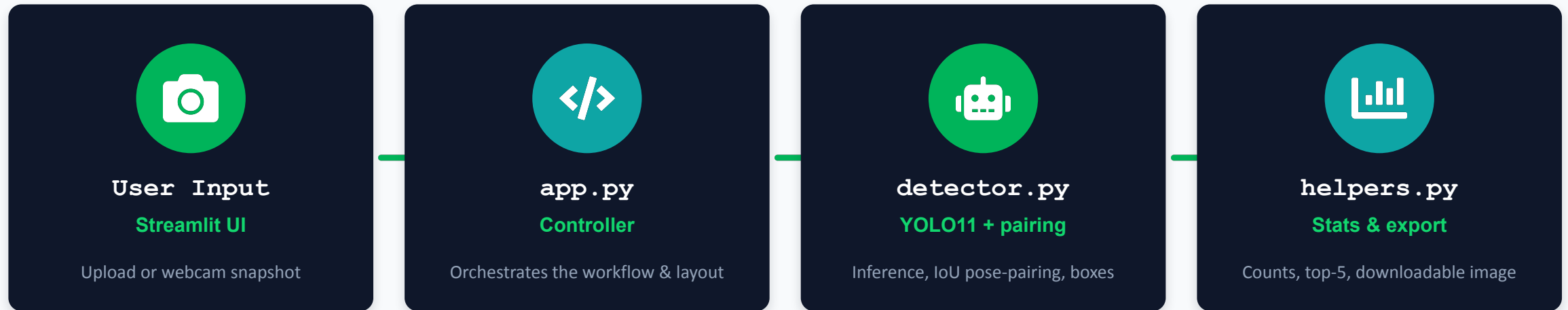
Pillow (PIL)

Reads uploads/webcam frames and exports the result image.

HOW IT FITS TOGETHER

System Architecture

Three small modules, one clear flow — from the user's image to a labelled result.



Output: annotated image + detection table + statistics + a one-click download of the result.

THE DETECTION ENGINE

Powered by YOLOv11

YOLO — “You Only Look Once” — is a real-time object detector that scans the whole image in a single pass, predicting every bounding box and class together. That single-pass design is what makes it fast enough for live use.



Single-pass

Looks at the image once — no slow region-by-region scanning.



Nano model

The smallest YOLOv11 variant: light, quick, easy to run on a laptop.



Fine-tuned

Re-trained on a custom animals dataset to learn animals + poses.

1

pass over the image

10

trained classes

0.25

default confidence

300

max detections / image

The Animals Dataset

The model is fine-tuned on the open-source Animals (animals-p1cov) dataset from Roboflow Universe — 10 classes split into animals and the poses they appear in.

5 Animals

 cat

 cow

 dog

 horse

 sheep

5 Poses

 eating

 laying

 run

 sit

 stand

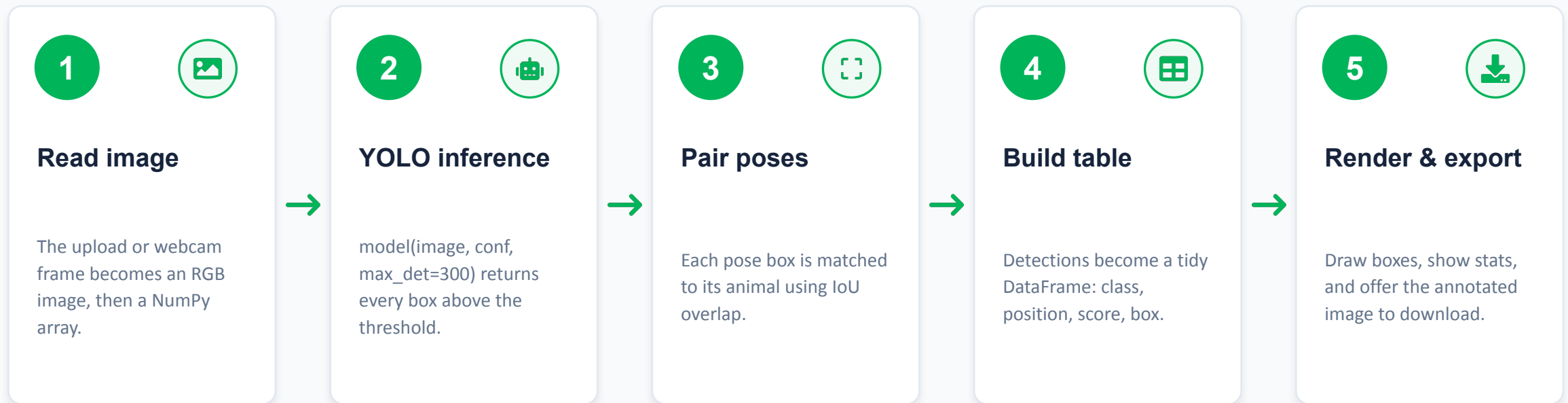


Dataset at a glance

Source	Roboflow Universe
Project	animals-p1cov · v3
Workspace	sesese
Format	YOLOv11
License	CC BY 4.0

How a Detection Happens

What runs each time you press “Run object detection”:



STEP 1 UNDER THE HOOD

Image to NumPy Array Conversion

How pixels translate into structural raw data for neural network consumption:



1. The Digital Input

An uploaded image consists of visual coordinates. The framework uses libraries like Pillow or OpenCV to read the raw compressed binary file into memory as an uncompressed RGB representation.



```
img_array = np.array(image.convert("RGB"))
```

Generates a structured multidimensional ndarray matching [Height, Width, Channels]

Red Channel

```
[[255, 120, ...]  
[ 84, 201, ...]  
[ 43, 92, ...]]
```

First matrix plane representing the intensity of red light per pixel (0-255).

Green Channel

```
[[102, 45, ...]  
[192, 88, ...]  
[ 12, 115, ...]]
```

Second matrix plane containing corresponding green value signals.

Blue Channel

```
[[ 50, 30, ...]  
[ 10, 15, ...]  
[240, 180, ...]]
```

Third matrix plane representing the blue color values.

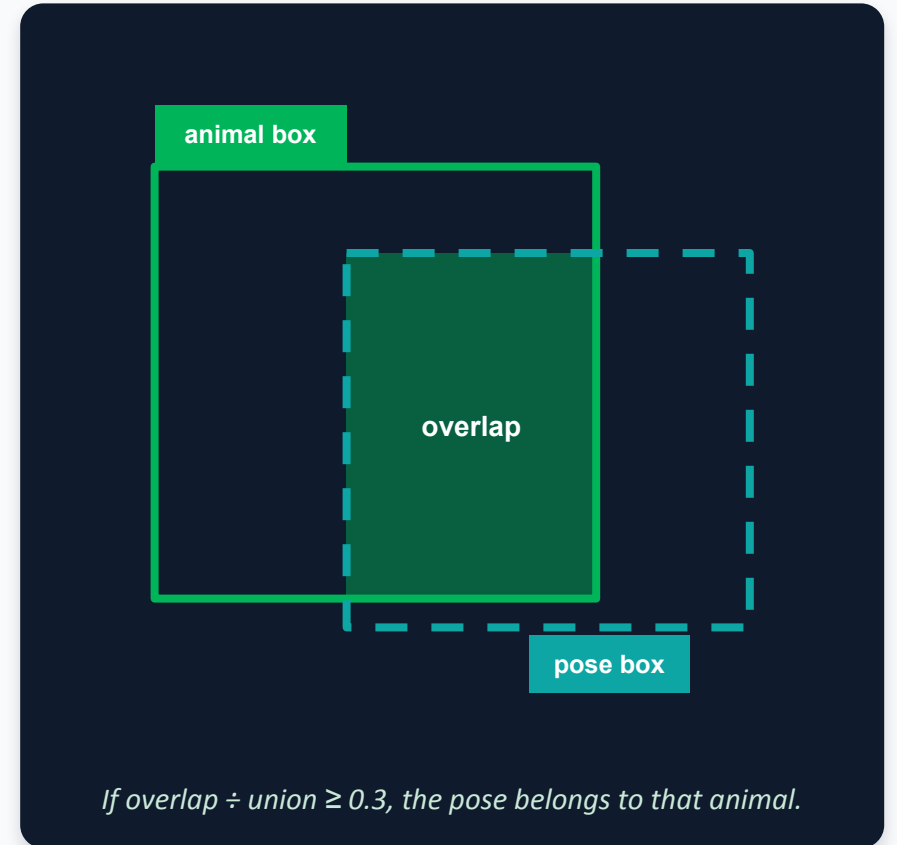
THE CLEVER PART

Pairing Poses with IoU

The model returns animals and poses as separate boxes. To say “cow (stand)”, VisionBox measures how much each pose box overlaps each animal box using Intersection over Union (IoU).

$$\text{IoU} = \text{Area of Overlap} / \text{Area of Union}$$

- ✓ **Highest overlap wins** — Each animal keeps the pose box it overlaps most.
- ✓ **Threshold = 0.3** — Overlaps below 0.3 IoU are ignored as too weak to pair.
- ✓ **Nothing is dropped** — Unmatched poses are still reported on their own row.



Inside the Code

visionbox/

└─ <code>app.py</code>	<i>Streamlit UI & workflow</i>
└─ <code>animals.pt</code>	<i>fine-tuned YOLOv11 weights</i>
└─ <code>utils/</code>	
└─ <code>detector.py</code>	<i>inference + IoU pairing</i>
└─ <code>helpers.py</code>	<i>stats & export</i>
└─ <code>requirements.txt</code>	<i>dependencies</i>
└─ <code>README.md</code>	<i>docs & workshop notes</i>



`detect_objects()`

Loads `animals.pt` once (cached) and runs YOLO11 on the image array.



`pair_detections()`

Computes IoU and merges each pose with the right animal box.



`draw_detections()`

Uses OpenCV to paint green boxes + “class (pose) score” labels.



`get_detection_stats()`

Unique classes, highest confidence and the top prediction.

Key Features



Upload or webcam

Drop in a JPG/PNG, or snap a photo straight from the browser camera.



Confidence slider

Tune the threshold live to keep only predictions you trust.



Boxes + poses

See green bounding boxes labelled with animal, pose and score.



Detection table

Class, position, confidence and exact box coordinates per object.



Instant statistics

Object count, unique classes, top prediction and the top-5 list.



Download result

Export the annotated image as a JPEG with a single click.

GETTING STARTED

Clone the Starter Template

To help you skip the boilerplate setup, a complete starter repository is ready on GitHub. Follow the steps below to clone it locally and begin the workshop.

```
git clone https://github.com/devMonkRahul/visionbox-workshop-starter-template
```

1

Clone & Navigate

Run the clone command in your terminal, then change your working directory into the project folder:

```
cd visionbox-workshop-starter-template
```

2

Install Dependencies

Set up your virtual environment and install all package requirements using uv or pip:

```
uv sync
```

or

```
pip install -r requirements.txt
```

3

Launch the App

Start the local Streamlit server to open the interactive workshop environment in your browser:

```
streamlit run app.py
```

From Photo to Result in 5 Steps

1

Add an image

Upload a file or capture a webcam snapshot in the app.

2

Set confidence

Drag the sidebar slider to your preferred threshold.

3

Run detection

Click “Run object detection” and let YOLOv11 work.

4

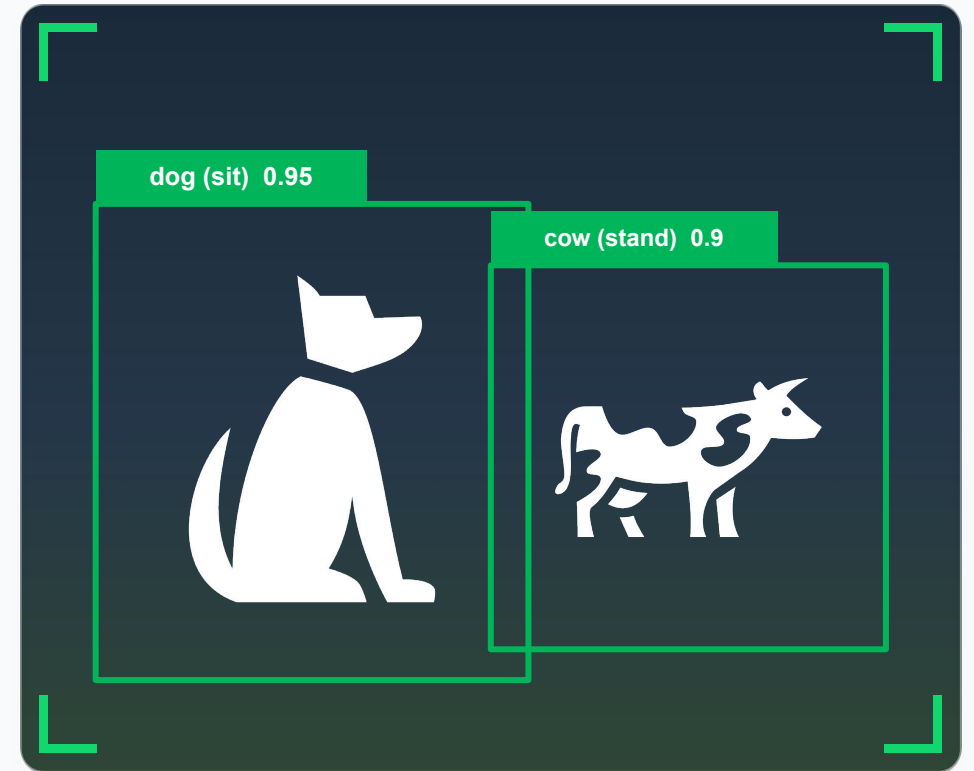
Read results

Inspect the annotated image, table and statistics.

5

Download

Save the labelled image to share or submit.



The annotated result, ready to download

What Students Take Away

VisionBox is a teaching tool first. By the end of the session, students can explain the ideas behind every result on screen.



What Computer Vision is

How machines turn pixels into meaning.



Classification vs detection

Why “where” matters as much as “what”.



Bounding boxes & labels

Reading box coordinates and class names.



Confidence scores

How a threshold filters predictions.



Multi-class labels

How animal + pose describe one subject.



Why fine-tuning helps

Adapting a general model to a new task.

Future Improvements

Where VisionBox can grow next:



Live video detection

Real-time streaming via a WebRTC Streamlit component.



Short video files

Run detection frame-by-frame on uploaded clips.



Detection history

Save past results so students can compare runs.



Class-frequency charts

Visualise how often each class appears.



Compare model sizes

Try different YOLOv11 variants side by side.



In-app quiz

A short quiz to reinforce workshop concepts.

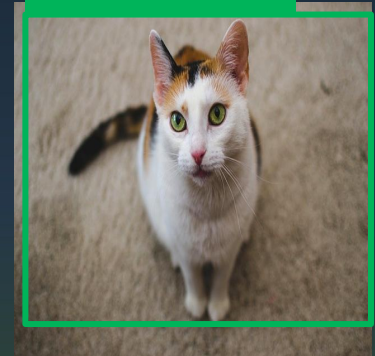
THANK YOU

See what your model sees.

VisionBox turns a single photo into labelled animals, their poses, and the confidence behind every call — a hands-on first step into Computer Vision.

`</>` github.com/devMonkRahul/VisionBox

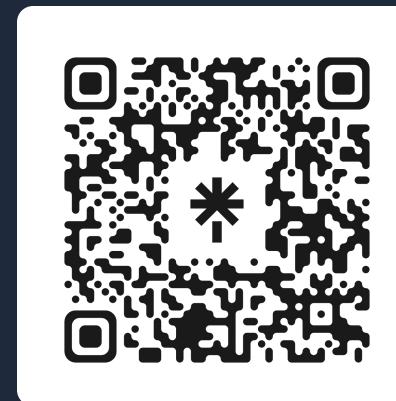
cat (sit) 0.97



LET'S CONNECT

Stay in Touch.

Scan the QR code to connect on professional platforms, explore the repository, or discuss computer vision developments and potential collaborations.



Scan to Connect